Bank Teller Discrete Event Simulation Report

Table of Content

1. Introduction.....1

2. High-Level Solution Strategy (Pseudocode).....1

2.1 One Teller Version.....1

2.2 Multi Teller Version (2, 3 and 4 Tellers).....2

3. Data Structures Used and Reasons.....3

4. Complexity Analysis (Big-O).....4

5. Program Execution and Output.....5

6. Simulation Outputs.....7

6.1 Teller Output.....7

6.2 Teller Output.....7

6.3 Teller Output.....8

6.4 Teller Output.....8

7. Discussion: Effect of Teller Number on Service Efficiency.....9

1. Introduction

This report describes the design and implementation of a discrete event driven bank teller queue simulation developed in Python. The program simulates customer arrival, service, waiting queues, and teller availability using different numbers of tellers.

The simulation was executed four times using 1, 2, 3, and 4 tellers respectively. A single priority queue is maintained for waiting customers. Customers are ordered first by priority level and then by arrival time.

2. High-Level Solution Strategy (Pseudocode)

2.1 One Teller Version

```
READ input filename

READ customer records until dummy record (0 0)

INITIALISE:
   one teller
   empty priority queue
   statistics
   simulation clock = 0

WHILE customers remain OR queue is not empty OR teller is busy

   DETERMINE next arrival time
   DETERMINE teller completion time

   IF completion happens first

      Move clock to completion time
      Mark teller idle

      IF queue is not empty
         Remove first priority customer
         Calculate waiting time
         Start service immediately
         Update completion time
      END IF
```

```
    ELSE

        Move clock to arrival time

        IF teller is idle
            Start customer service immediately
        ELSE
            Insert customer into priority queue
            Update maximum queue length
        END IF

    END IF

END WHILE

CALCULATE statistics

PRINT results
```

2.2 Multi Teller Version (2, 3 and 4 Tellers)

```
READ number of tellers

READ input filename

READ customer records until dummy record (0 0)

INITIALISE:
    multiple tellers
    one empty priority queue
    Round Robin pointer
    statistics
    simulation clock = 0

WHILE customers remain OR queue is not empty OR any teller is busy

    FIND next arrival time
    FIND earliest teller completion time

    IF completion happens first

        Move clock to completion time
```

```
        Mark completed teller idle

        IF queue is not empty
            Remove first priority customer
            Calculate waiting time
            Assign customer to teller
            Update completion time
        END IF

    ELSE

        Move clock to arrival time
        FIND idle teller using Round Robin

        IF idle teller exists
            Assign customer to teller
        ELSE
            Insert customer into priority queue
            Update maximum queue length
        END IF

    END IF

END WHILE

CALCULATE required statistics

PRINT results
```

3. Data Structures Used and Reasons

Customer List: a python list that holds the customer records. Each record contains the arrival time, the service time, the priority and the order of the customer. It allows sequential reading of customers from the input file.

Priority Queue : List maintained manually to maintain waiting queue The insertion order is first by priority, then by time of arrival. This follows the assignment requirement of implementing data structures without using external queue libraries.

Teller List (Multi Teller): A list of teller objects that keeps track of the status of each teller (whether they are busy, when they will be finished, how many customers they have served, and how long they were busy).

Round Robin Pointer : The next teller position to check is stored in an integer variable so that the work is evenly distributed between the available tellers.

4. Complexity Analysis (Big-O)

Let n represent the number of customers and q represent the maximum queue length.

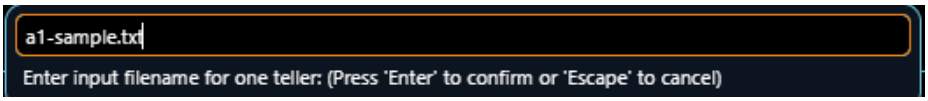
Reading the input file requires O(n). Inserting a customer into the manually ordered priority queue requires O(q), while removing the first queue customer requires O(q) because elements are shifted. Therefore, the simulation requires O(nq) time, which becomes $O(n^2)$ in the worst case when q grows proportionally with n.

For 2, 3 and 4 teller simulations, searching for an available teller using Round Robin requires O(s), where s is the number of tellers. Since s is fixed and small (2, 3, or 4), this remains constant. The global worst-case complexity is still $O(n^2)$

The space complexity is O(n). We need to keep the customer records and the waiting queue both dependent on the number of customers.

5. Program Execution and Output

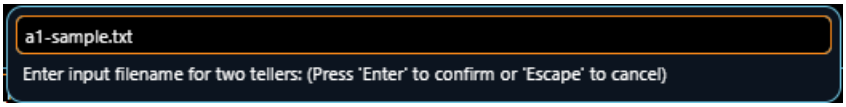
The program was executed using the provided a1-sample.txt input file. Four simulations were performed using 1, 2, 3, and 4 tellers.



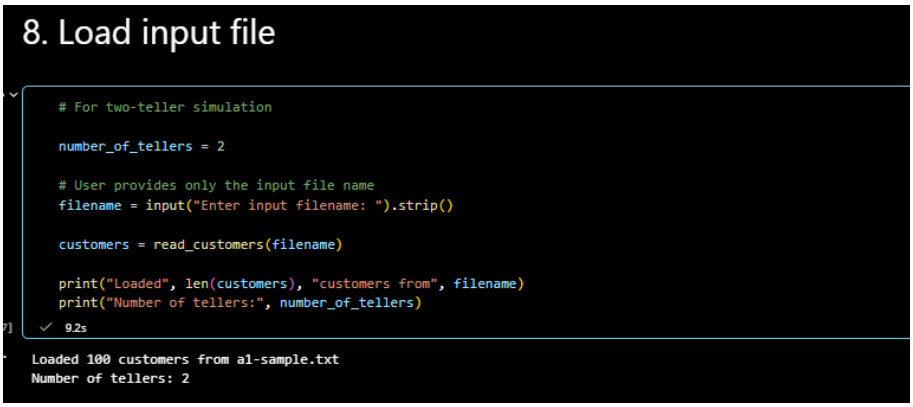
Screenshot 1: One teller input file loading and program run



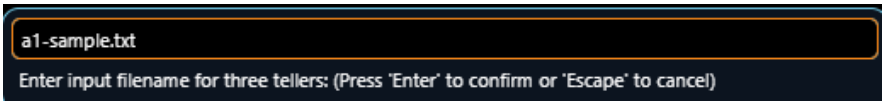
Screenshot 2: One teller simulation output



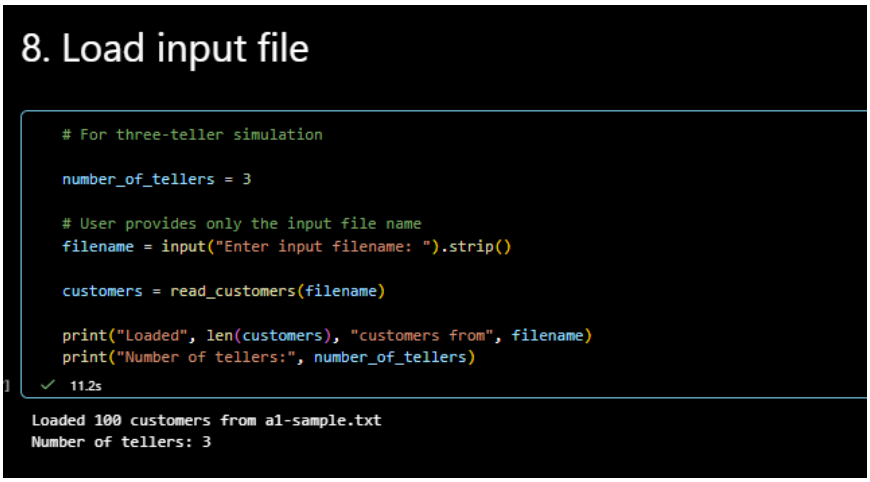
Screenshot 3: Two teller input file loading and program run



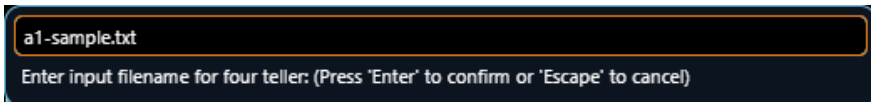
Screenshot 4: Two teller simulation output



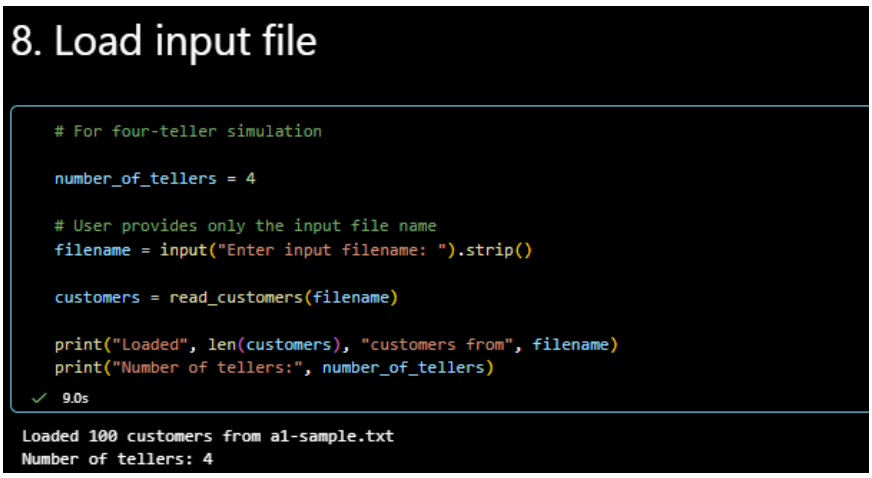
Screenshot 5: Three teller input file loading and program run



Screenshot 6: Three teller simulation output



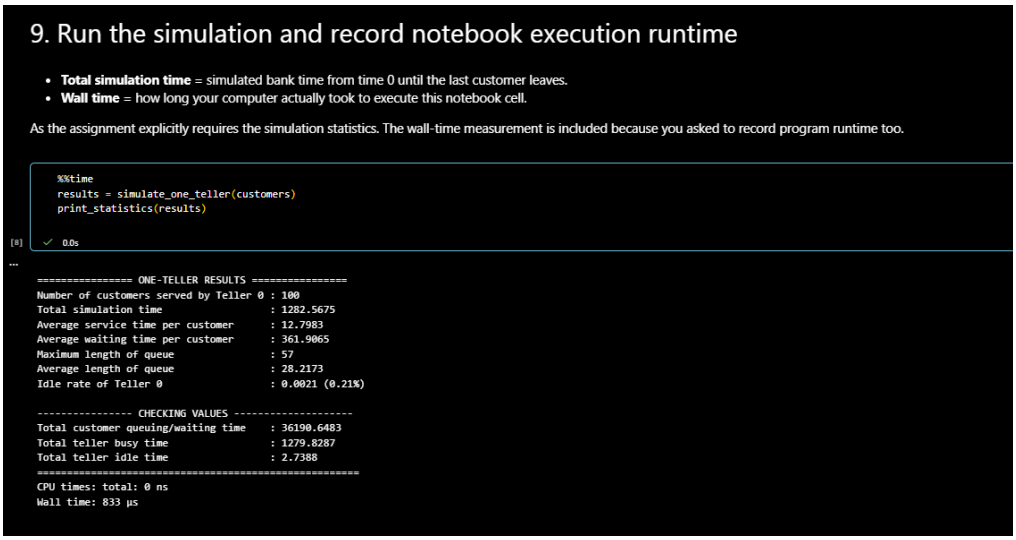
Screenshot 7: Four teller input file loading and program run



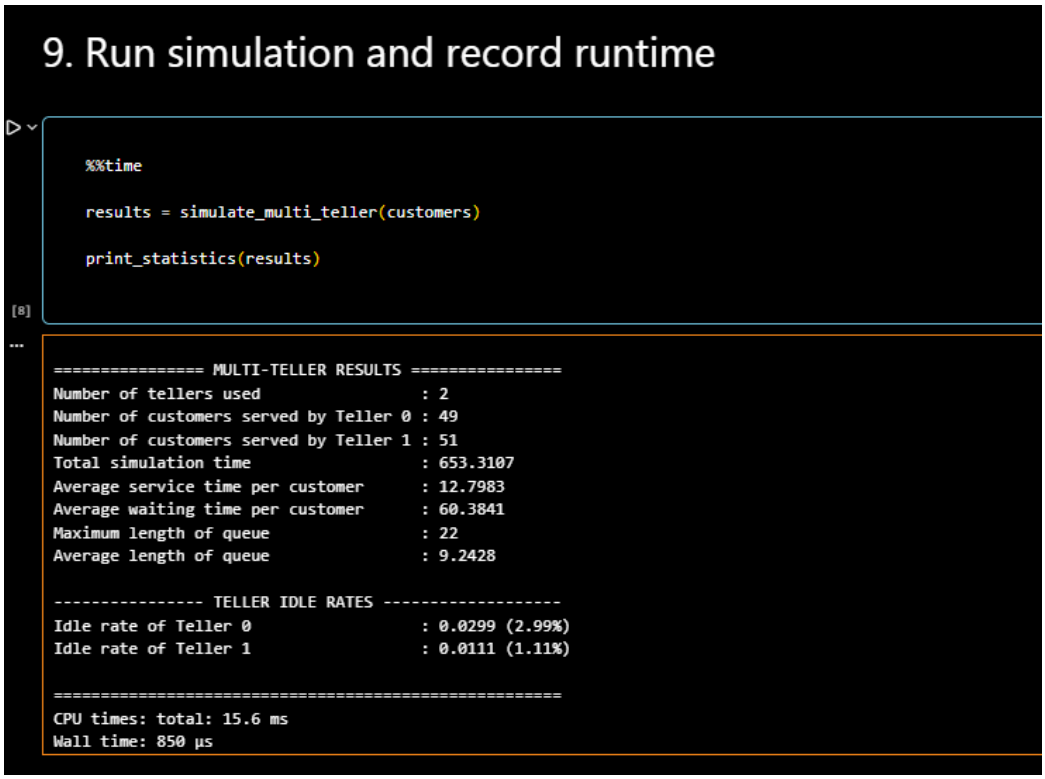
Screenshot 8: Four teller simulation output

6. Simulation Outputs

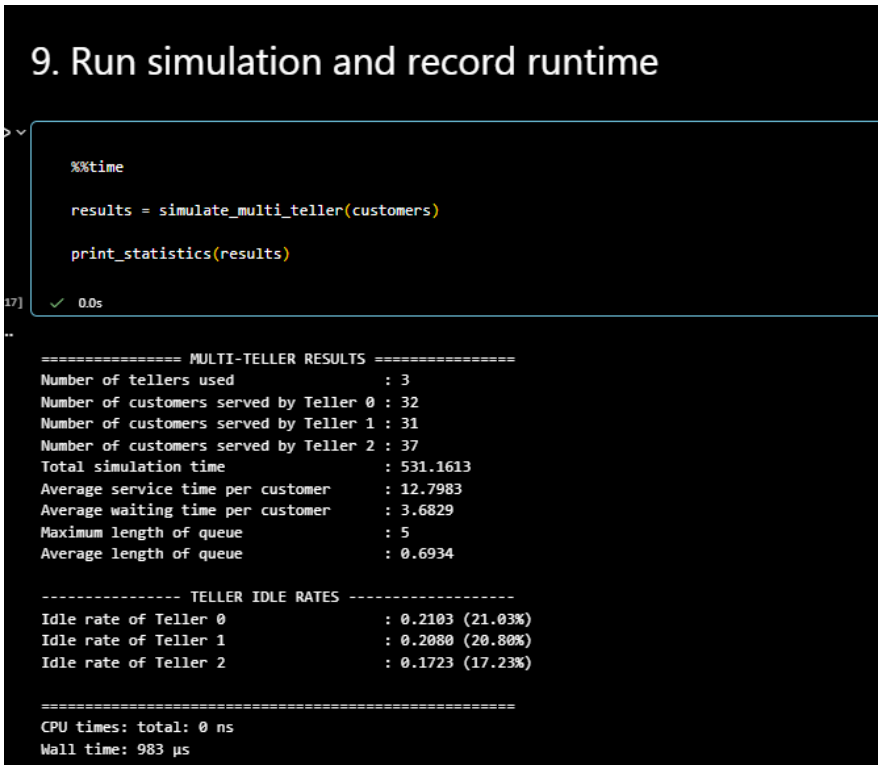
6.1 Teller Output



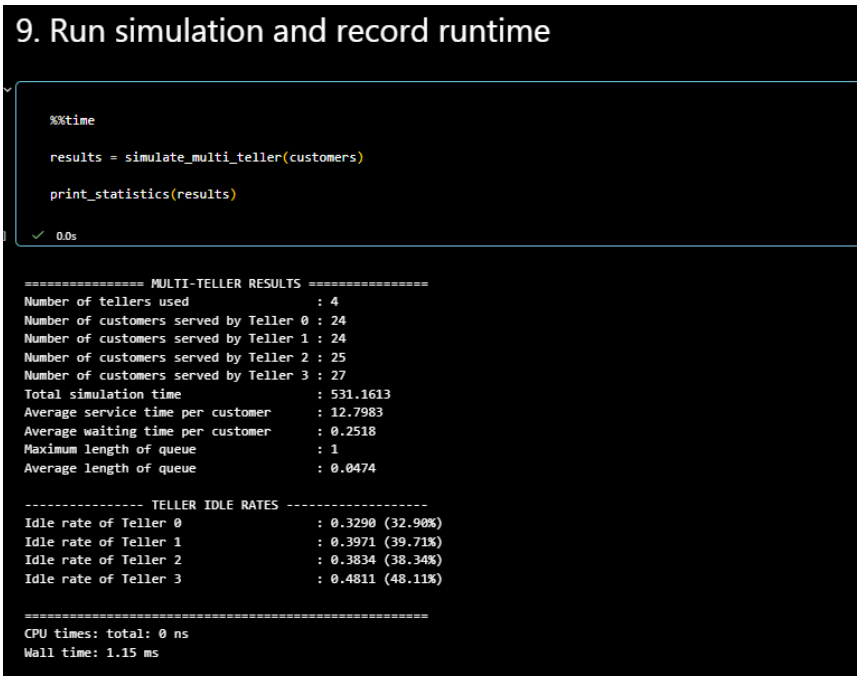
6.2 Teller Output



6.3 Teller Output



6.4 Teller Output



7. Discussion: Effect of Teller Number on Service Efficiency

The number of tellers dictates the length of queues and how long customers must wait. More tellers usually allow more customers to be served at the same time and thus reduce the time customers spend waiting in the queue. But more tellers also mean more time spent idle when there are fewer customers.

The comparison should be based on the statistics generated such as average waiting time, maximum queue length, average queue length and teller idle rates.